

TypeScript and React

Basic to Advanced

Language, Hooks, State, Performance, Patterns
Detailed Comparison, Examples and Syntax

Contents

How to Use This Guide	5
The 3-volume series	5
Why TypeScript and React are taught together	5
Versions this guide targets	5
Study plan	5
1. JavaScript Essentials Before TypeScript	6
1.1 Variables and scope	6
1.2 Functions	6
1.3 Destructuring, spread and optional chaining	6
1.4 Array methods you must know	6
1.5 Async, modules and classes	7
1.6 Common JavaScript gotchas	7
2. TypeScript - Basics	8
2.1 What is TypeScript?	8
2.2 Setup	8
2.3 Primitive types and inference	8
2.4 Arrays, tuples, objects	8
2.5 Type aliases vs interfaces	9
2.6 Union, literal and enum types	9
2.7 Functions	9
2.8 Classes	10
2.9 Generics (introduction)	10
2.10 tsconfig.json essentials	11
2.11 TypeScript vs JavaScript	11
3. TypeScript - Intermediate	13
3.1 Type narrowing	13
3.2 Discriminated unions (the most useful pattern)	13
3.3 Built-in utility types	14
3.4 keyof, typeof, indexed access	14
3.5 Generics in depth	14
3.6 Mapped types and conditional types	15
3.7 Modules and declaration files	15
3.8 Typing asynchronous code and APIs	16
3.9 Error handling patterns	16
4. TypeScript - Advanced	17
4.1 Template literal types	17

4.2 The satisfies operator	17
4.3 Branded (nominal) types	17
4.4 Advanced generic patterns	17
4.5 Variance and assignability (why some assignments fail)	18
4.6 Decorators (standard TC39 in TS 5)	18
4.7 The using keyword (explicit resource management)	19
4.8 Strictness and safety checklist	19
4.9 Performance and project organisation	19
4.10 Running TypeScript directly	19
4.11 TypeScript vs Java (for backend developers)	19
5. React - Basics	21
5.1 What is React and why use it?	21
5.2 Create a project (Vite + React + TypeScript)	21
5.3 JSX	22
5.4 Components and props	22
5.5 State with useState	23
5.6 Events	23
5.7 Conditional rendering and lists	24
5.8 Lifting state up and thinking in React	24
6. React - Hooks in Depth	25
6.1 The rules of hooks	25
6.2 useEffect: synchronising with the outside world	25
6.3 useRef: mutable value or DOM handle	26
6.4 useContext: avoid prop drilling	26
6.5 useReducer: complex state logic	26
6.6 useMemo and useCallback: performance hints	27
6.7 Other built-in hooks	27
6.8 React 19 hooks and APIs	27
6.9 Custom hooks: reuse logic, not UI	28
6.10 Hooks cheat sheet	29
7. React - Intermediate	30
7.1 Forms: controlled vs uncontrolled	30
7.2 Data fetching: TanStack Query (server state)	31
7.3 Routing with React Router	31
7.4 State management: choosing the right tool	32
7.5 Styling options	33
7.6 Typing React components	33
8. React - Advanced	35

8.1 How rendering really works	35
8.2 Performance optimisation playbook	35
8.3 Error boundaries and Suspense	36
8.4 Component design patterns	37
8.5 Project structure and architecture	37
8.6 Testing React	37
8.7 Accessibility (a11y) essentials	38
8.8 React Server Components (RSC) - the next paradigm	38
8.9 React Compiler	39
8.10 Common React mistakes	39
9. Comparisons: UI Frameworks and Rendering Strategies	40
9.1 React vs Angular vs Vue vs Svelte	40
9.2 Same component in four frameworks (counter)	41
9.3 Rendering strategies	41
9.4 State management: quick comparison (React ecosystem)	42
9.5 Build tools comparison	42
9.6 Language-level comparison for full-stack developers	42
10. Full Example: Task Manager UI (TypeScript + React)	43
10.1 Install	43
10.2 Types and API layer (shared contract)	43
10.3 Query hooks	44
10.4 Components	44
10.5 What this example demonstrates	45
11. Interview Q&A, Cheat Sheet and Roadmap	46
11.1 TypeScript interview questions	46
11.2 React interview questions	46
11.3 Cheat sheet: TypeScript syntax	47
11.4 Cheat sheet: React patterns	47
11.5 Learning roadmap	48

How to Use This Guide

CATEGORY

Volume 2 of 3 - Language and UI. Covers TypeScript (the typed layer over JavaScript) and React (the component library for building user interfaces), from beginner to advanced, with comparisons to alternatives. Level: Beginner to Advanced.

The 3-volume series

Volume	Topics	Category
Vol 1 - Backend	Spring Boot, Node.js, Express, NestJS	Server-side / API
Vol 2 - Language and UI (this file)	TypeScript, React	Language + client-side UI
Vol 3 - Full-stack and Styling	Next.js, Tailwind CSS, architecture comparison	Framework + styling

Why TypeScript and React are taught together

TypeScript catches mistakes while you type; React turns UI into small reusable functions. Modern React projects are written in TypeScript by default, and the same TypeScript types can be shared with your Node.js or Next.js backend (Volume 1 and 3).

Versions this guide targets

Technology	Version	Notes
TypeScript	5.x	<code>satisfies</code> , <code>const</code> type parameters, decorators (standard), <code>using</code>
React	19.x	Actions, <code>use</code> , <code>useActionState</code> , <code>useOptimistic</code> , ref as prop
Vite	6/7	Standard dev server and bundler for plain React
React Router	7	Also usable as a framework
TanStack Query	5	Server-state management
Vitest + Testing Library	current	Component testing

Study plan

- Week 1:** TypeScript basics (Ch 1-2) and JavaScript refresher.
- Week 2:** TypeScript intermediate and advanced (Ch 3-4).
- Week 3:** React basics and hooks (Ch 5-6).
- Week 4:** React intermediate and advanced (Ch 7-8), then the comparison, full example and revision (Ch 9-11).

1. JavaScript Essentials Before TypeScript

CATEGORY

Foundation. The modern JavaScript (ES2015+) features that TypeScript and React rely on every day.
Level: Beginner.

1.1 Variables and scope

JS

```
const PI = 3.14;           // cannot be reassigned (use by default)
let count = 0;             // can be reassigned, block scoped
// var                     // function scoped, hoisted - avoid in new code

const user = { name: "Asha", age: 25 };
user.age = 26;             // allowed: const protects the binding, not the object contents
```

1.2 Functions

JS

```
function add(a, b) { return a + b; }           // declaration (hoisted)
const addArrow = (a, b) => a + b;              // arrow function (lexical this)
const greet = (name = "friend") => `Hello ${name}`; // default parameter + template string
const sum = (...nums) => nums.reduce((t, n) => t + n, 0); // rest parameters
```

1.3 Destructuring, spread and optional chaining

JS

```
const { name, age = 18, address: { city } = {} } = user; // object destructuring with defaults
const [first, , third, ...others] = [1, 2, 3, 4, 5];      // array destructuring

const merged = { ...user, age: 30 };                     // copy + override (immutable update)
const withItem = [...items, newItem];                    // new array
const without = items.filter(i => i.id !== id);

user?.address?.city; // undefined instead of an error if a link is missing
const label = user.nickname ?? "No nickname"; // ?? only for null/undefined (|| also catches 0 and "")
```

1.4 Array methods you must know

Method	Purpose	Example
map	Transform every item	<code>[1,2,3].map(n => n * 2)</code> gives <code>[2,4,6]</code>
filter	Keep matching items	<code>xs.filter(x => x > 1)</code>
find	First match or <code>undefined</code>	<code>users.find(u => u.id === 3)</code>
some / every	Any / all match	<code>xs.some(x => x < 0)</code>
reduce	Fold into one value	<code>xs.reduce((t, x) => t + x, 0)</code>
flatMap	Map then flatten	<code>xs.flatMap(x => [x, x])</code>
sort	Sort in place	<code>[...xs].sort((a, b) => a - b)</code>
toSorted, toReversed	Non-mutating (ES2023)	<code>xs.toSorted()</code>
Object.entries	Object to <code>[key, value]</code> pairs	<code>Object.entries(o).map(([k, v]) => ...)</code>
Object.groupBy	Group items (ES2024)	<code>Object.groupBy(xs, x => x.type)</code>

1.5 Async, modules and classes

JS

```
// Promises and async/await
const res = await fetch("/api/tasks");
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const data = await res.json();

// Modules
export const add = (a, b) => a + b;           // named export
export default function App() {}           // default export
import App, { add } from "./math.js";

// Classes
class Animal {
  #secret = 42;                             // private field
  constructor(name) { this.name = name; }
  speak() { return `${this.name} makes a sound`; }
  static create(n) { return new Animal(n); }
}
class Dog extends Animal { speak() { return `${super.speak()} - Woof`; } }
```

1.6 Common JavaScript gotchas

Gotcha	Example	Fix
Loose equality	<code>0 == "0"</code> is true	Use <code>===</code>
<code>this</code> changes	Method passed as callback loses <code>this</code>	Arrow functions or <code>.bind</code>
Mutation	<code>sort()</code> and <code>push()</code> modify the original	Copy first <code>[...xs]</code>
Falsy values	<code>0</code> , <code>""</code> , <code>null</code> , <code>undefined</code> , <code>NaN</code> , <code>false</code>	Use <code>??</code> and explicit checks
Floating point	<code>0.1 + 0.2 !== 0.3</code>	Work in integers (cents) or use a decimal library
<code>typeof null</code>	Returns <code>"object"</code>	Check <code>x === null</code>

2. TypeScript - Basics

CATEGORY

Language: static types for JavaScript. Type annotations, interfaces, unions, generics, classes and configuration. Used in virtually all serious frontend and Node.js projects. Level: Beginner.

2.1 What is TypeScript?

TypeScript is a **superset of JavaScript**: every valid JavaScript program is valid TypeScript. It adds a **type system** that is checked at compile time, then the types are **erased** and plain JavaScript runs. TypeScript never changes runtime behaviour and adds no runtime cost.

TEXT

```
.ts / .tsx ---- tsc / esbuild / swc / tsx ----> .js
(types checked)                        (types erased, runs in browser/Node)
```

Benefits: autocomplete, safe refactoring, self-documenting code, early bug detection (typos, **undefined** errors), shared types between frontend and backend.

2.2 Setup

BASH

```
npm install -D typescript
npx tsc --init           # creates tsconfig.json
npx tsc                  # type-check + compile
npx tsc --noEmit         # type-check only (what bundlers usually rely on)
npx tsx script.ts        # run directly (dev convenience)
```

2.3 Primitive types and inference

TS

```
let title: string = "Learn TS";
let count: number = 42;           // integers and floats are both number
let done: boolean = false;
let big: bigint = 9007199254740993n;
let id: symbol = Symbol("id");
let nothing: null = null;
let notSet: undefined = undefined;
let inferred = "hello";          // TypeScript infers: string (annotation not needed)
const fixed = "hello";           // inferred as the literal type "hello"

let anything: any = 5;            // disables checking - AVOID
let safe: unknown = JSON.parse("1"); // must narrow before use - PREFER over any
if (typeof safe === "number") safe.toFixed(2);

function fail(msg: string): never { throw new Error(msg); } // never returns
function log(msg: string): void { console.log(msg); }        // returns nothing
```

2.4 Arrays, tuples, objects

TS

```
const nums: number[] = [1, 2, 3];
const names: Array<string> = ["a", "b"];
const pair: [string, number] = ["age", 30]; // tuple: fixed length and types
const point: readonly [x: number, y: number] = [1, 2]; // labelled readonly tuple

const user: { id: number; name: string; email?: string; readonly role: string } = {
  id: 1, name: "Asha", role: "admin",
}; // ? = optional, readonly = cannot reassign
```

2.5 Type aliases vs interfaces

TS

```

type Id = string | number;           // alias for any type (unions, tuples, primitives...)
type Point = { x: number; y: number };

interface User {                     // describes object shapes; can be extended and merged
  id: Id;
  name: string;
  greet(): string;
}
interface Admin extends User { permissions: string[] }
type Manager = User & { reports: User[] }; // intersection: has everything from both

interface Window { myGlobal: string } // declaration merging: reopen an interface (interfaces only)

```

Feature	type	interface
Object shapes	Yes	Yes
Unions, tuples, primitives	Yes	No
Extend	Intersection &	extends
Declaration merging	No	Yes
Performance in huge codebases	Slightly slower for deep intersections	Slightly faster (cached)
Implemented by classes	Yes (object types)	Yes

PRO TIP

Practical rule: use **interface** for public object contracts and class shapes, **type** for unions, mapped/conditional types and everything else. Consistency in a team matters more than the choice.

2.6 Union, literal and enum types

TS

```

type Status = "OPEN" | "IN_PROGRESS" | "DONE";           // string literal union (most common)
type Priority = 1 | 2 | 3 | 4 | 5;
let s: Status = "OPEN";
// s = "CLOSED"; // Error: not assignable to type Status

function pad(v: string | number) {
  return typeof v === "number" ? v.toString().padStart(3, "0") : v; // narrowing
}
enum Direction { Up, Down, Left, Right }                // compiles to a runtime object
const enum Flag { A = 1, B = 2 }                         // inlined at compile time (careful with tools)

```

Enum alternative	Code	Why
Literal union	<code>type Role = "admin" "user"</code>	Zero runtime code, works everywhere
as const object	<code>const Role = { Admin: "admin", User: "user" } as const; type Role = typeof Role[keyof typeof Role];</code>	Runtime values + type from one source

2.7 Functions

TS

```
function add(a: number, b: number): number { return a + b; }
const mul = (a: number, b: number): number => a * b;

function greet(name: string, greeting = "Hello", punctuation?: string): string {
  return `${greeting}, ${name}${punctuation ?? "!"}`;
}

type Handler = (event: MouseEvent) => void;           // function type
type Compare<T> = (a: T, b: T) => number;

// Overloads: several call signatures, one implementation
function parse(v: string): number;
function parse(v: string[]): number[];
function parse(v: string | string[]) { return Array.isArray(v) ? v.map(Number) : Number(v); }

// `this` typing and rest parameters
function total(label: string, ...values: number[]): string { return `${label}: ${values.length}`; }
```

2.8 Classes

TS

```
class Account {
  static count = 0;
  readonly id: number;
  private balance = 0;           // TypeScript-only privacy
  #pin = "1234";                 // real JavaScript private field (runtime)

  constructor(public owner: string, initial = 0) { // parameter property: declares + assigns
    this.id = ++Account.count;
    this.balance = initial;
  }

  deposit(amount: number): this {
    if (amount <= 0) throw new RangeError("amount must be positive");
    this.balance += amount;
    return this;                 // fluent API
  }
  get total(): number { return this.balance; }      // getter
}

abstract class Shape { abstract area(): number; describe() { return `Area ${this.area()}`; } }
class Circle extends Shape { constructor(private r: number) { super(); } area() { return Math.PI * this.r ** 2; } }

interface Printable { print(): void }
class Report implements Printable { print() { console.log("report"); } }
```

Modifier	Meaning
public (default)	Accessible anywhere
private	Only inside the class (compile-time only)
protected	Class and subclasses
readonly	Assign once (declaration or constructor)
#field	Truly private at runtime

2.9 Generics (introduction)

Generics let you write code that works with **any type while preserving type information**.

TS

```
function first<T>(items: T[]): T | undefined { return items[0]; }
const a = first([1, 2, 3]);           // a: number | undefined
const b = first(["x", "y"]);          // b: string | undefined

interface ApiResponse<T> { data: T; status: number; error?: string }
type UserResponse = ApiResponse<{ id: number; name: string }>;

class Box<T> { constructor(public value: T) {} map<U>(fn: (v: T) => U): Box<U> { return new
  Box(fn(this.value)); } }

function pick<T, K extends keyof T>(obj: T, key: K): T[K] { return obj[key]; } // constraint
pick({ id: 1, name: "A" }, "name");    // string (pick(..., "zzz") is a compile error)
```

2.10 tsconfig.json essentials

JSON

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "lib": ["ES2023", "DOM", "DOM.Iterable"],
    "jsx": "react-jsx",
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "noImplicitOverride": true,
    "exactOptionalPropertyTypes": false,
    "skipLibCheck": true,
    "isolatedModules": true,
    "resolveJsonModule": true,
    "esModuleInterop": true,
    "noEmit": true,
    "baseUrl": ".",
    "paths": { "@/*": [".src/*"] }
  },
  "include": ["src"]
}
```

Option	What it does
<code>strict</code>	Turns on all strict checks (<code>strictNullChecks</code> , <code>noImplicitAny</code> , ...). Always enable.
<code>noUncheckedIndexedAccess</code>	<code>arr[0]</code> becomes <code>T undefined</code> , forcing safe access
<code>jsx: "react-jsx"</code>	Modern JSX transform (no <code>import React</code> needed)
<code>moduleResolution: "Bundler"</code>	Matches Vite / Next.js behaviour
<code>isolatedModules</code>	Each file must be compilable alone (needed for esbuild/swc/Babel)
<code>paths</code>	Import aliases such as <code>@/components/Button</code>
<code>skipLibCheck</code>	Faster builds by skipping type-checks of <code>.d.ts</code> files
<code>noEmit</code>	Only check types; the bundler produces JS

2.11 TypeScript vs JavaScript

Aspect	JavaScript	TypeScript
Type checking	Runtime only	Compile time (plus runtime is still JS)
Error discovery	When code runs	While writing / in CI
Tooling	Basic autocomplete	Rich autocomplete, rename, go-to-definition, safe refactors
Refactoring large code	Risky	Safe
Setup	None	tsconfig + build step (small)

Aspect	JavaScript	TypeScript
Runtime overhead	None	None (types erased)
Learning curve	Lower	Extra concepts (generics, utility types)
Best for	Scripts, tiny projects	Applications, libraries, teams

3. TypeScript - Intermediate

CATEGORY

Language: everyday power features. Narrowing, type guards, discriminated unions, utility types, mapped types, modules, declaration files and typing async code and APIs. Level: Intermediate.

3.1 Type narrowing

TypeScript refines a broad type to a narrower one inside branches, based on checks it understands.

TS

```
function format(v: string | number | Date | null | undefined) {
  if (v == null) return "-"; // removes null and undefined
  if (typeof v === "string") return v.trim(); // string here
  if (typeof v === "number") return v.toFixed(2); // number here
  return v.toISOString(); // Date here
}

function len(x: string | string[]) { return Array.isArray(x) ? x.length : x.length; }
function area(s: Circle | Square) { return "radius" in s ? Math.PI * s.radius ** 2 : s.side ** 2; }
function when(e: Date | string) { return e instanceof Date ? e.getTime() : Date.parse(e); }
```

Custom type guards and assertion functions

TS

```
interface Cat { meow(): void }
interface Dog { bark(): void }

function isCat(p: Cat | Dog): p is Cat { return "meow" in p; } // type predicate
function assertString(v: unknown): asserts v is string { // throws if not
  if (typeof v !== "string") throw new TypeError("expected string");
}

const raw: unknown = getInput();
assertString(raw);
raw.toUpperCase(); // raw is string from here on
```

3.2 Discriminated unions (the most useful pattern)

A shared literal property (the *discriminant*) lets TypeScript know exactly which variant you have.

TS

```
type LoadState<T> =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "success"; data: T }
  | { status: "error"; error: Error };

function render(s: LoadState<User[]>) {
  switch (s.status) {
    case "idle": return "Press load";
    case "loading": return "Loading...";
    case "success": return `${s.data.length} users`; // s.data exists only here
    case "error": return s.error.message;
    default: {
      const _exhaustive: never = s; // compile error if a case is missing
      return _exhaustive;
    }
  }
}
```

This is the TypeScript equivalent of Java sealed interfaces + pattern matching `switch`, and it makes "impossible states" unrepresentable in React state.

3.3 Built-in utility types

Utility	Meaning	Example
<code>Partial<T></code>	All properties optional	<code>Partial<User></code> for update payloads
<code>Required<T></code>	All properties required	<code>Required<Config></code>
<code>Readonly<T></code>	All properties readonly	<code>Readonly<State></code>
<code>Pick<T, K></code>	Keep only listed keys	<code>Pick<User, "id" "name"></code>
<code>Omit<T, K></code>	Remove listed keys	<code>Omit<User, "password"></code>
<code>Record<K, V></code>	Object with keys K and values V	<code>Record<Status, number></code>
<code>Exclude<U, X></code>	Remove members from a union	<code>Exclude<"a" "b", "a"></code> gives "b"
<code>Extract<U, X></code>	Keep members of a union	<code>Extract<Event, { type: "click" }></code>
<code>NonNullable<T></code>	Remove null and undefined	<code>NonNullable<string null></code>
<code>ReturnType<F></code>	Return type of a function	<code>ReturnType<typeof fetchUser></code>
<code>Parameters<F></code>	Tuple of parameter types	<code>Parameters<typeof add></code>
<code>Awaited<T></code>	Unwrap a Promise	<code>Awaited<Promise<User>></code> is <code>User</code>
<code>InstanceType<C></code>	Instance type of a class	<code>InstanceType<typeof Account></code>

TS

```
interface User { id: number; name: string; email: string; password: string }

type PublicUser = Omit<User, "password">;
type UserUpdate = Partial<Omit<User, "id">>;
type UserPreview = Pick<User, "id" | "name">;
type ByStatus = Record<"OPEN" | "DONE", User[]>;
type LoginResult = Awaited<ReturnType<typeof login>>;
```

3.4 keyof, typeof, indexed access

TS

```
const config = { host: "localhost", port: 3000, debug: true } as const;

type Config = typeof config;           // derive a type from a value
type ConfigKey = keyof Config;         // "host" | "port" | "debug"
type Port = Config["port"];            // 3000 (literal because of as const)

const ROLES = ["admin", "editor", "viewer"] as const;
type Role = typeof ROLES[number];      // "admin" | "editor" | "viewer" (one source of truth)
```

3.5 Generics in depth

TS

```
// Constraints
function longest<T extends { length: number }>(a: T, b: T): T { return a.length >= b.length ? a : b; }

// Default type parameter
interface Page<T = unknown> { content: T[]; total: number }

// Generic functions returning inferred types
function groupBy<T, K extends PropertyKey>(items: T[], key: (i: T) => K): Record<K, T[]> {
  return items.reduce((acc, item) => {
    (acc[key(item)] ||= []).push(item);
    return acc;
  }, {} as Record<K, T[]>);
}
const byStatus = groupBy(tasks, t => t.status); // Record<Status, Task[]>

// Generic React-style hook signature
function useLocalStorage<T>(key: string, initial: T): [T, (v: T) => void] { /* ... */ }
```

3.6 Mapped types and conditional types

TS

```
// Mapped type: transform every property
type Nullable<T> = { [K in keyof T]: T[K] | null };
type Getters<T> = { [K in keyof T as `get${Capitalize<string & K>}`]: () => T[K] };
type ReadonlyDeep<T> = { readonly [K in keyof T]: T[K] extends object ? ReadonlyDeep<T[K]> : T[K] };

// Conditional type: if / else at the type level
type IsString<T> = T extends string ? "yes" : "no";
type A = IsString<"x">; // "yes"
type B = IsString<42>; // "no"

// Distributes over unions
type ToArray<T> = T extends any ? T[] : never;
type C = ToArray<string | number>; // string[] | number[]

// infer: extract a type from inside another
type ElementOf<T> = T extends (infer E)[] ? E : never;
type UnwrapPromise<T> = T extends Promise<infer U> ? U : T;
type Fn1 = ElementOf<string[]>; // string
```

3.7 Modules and declaration files

TS

```
// math.ts
export interface Vec { x: number; y: number }
export const add = (a: Vec, b: Vec): Vec => ({ x: a.x + b.x, y: a.y + b.y });
export default class Calculator {}

// use.ts
import Calculator, { add, type Vec } from "./math"; // `type` import is erased at build time
import type { Task } from "@types"; // only a type: guaranteed no runtime import
export type { Vec }; // re-export a type
```

- **.d.ts** files describe the types of JavaScript libraries. Install **@types/node**, **@types/express** and similar from DefinitelyTyped (**npm i -D @types/xyz**). Many modern libraries ship their own types.
- **declare module "*.svg" { const src: string; export default src; }** teaches TypeScript about non-code imports.
- **declare global { interface Window { analytics: Analytics } }** extends global types.

3.8 Typing asynchronous code and APIs

TS

```
interface Task { id: number; title: string; status: "OPEN" | "DONE" }

async function fetchJson<T>(url: string, init?: RequestInit): Promise<T> {
  const res = await fetch(url, init);
  if (!res.ok) throw new HttpError(res.status, await res.text());
  return res.json() as Promise<T>; // NOTE: `as` is a promise, not a proof! (see below)
}

const tasks = await fetchJson<Task[]>("/api/tasks");
const [a, b] = await Promise.all([fetchJson<User>("/u"), fetchJson<Order[]>("/o")]); // typed tuple

class HttpError extends Error {
  constructor(public status: number, message: string) { super(message); this.name = "HttpError"; }
}
```

WARNING

TypeScript types disappear at runtime. `res.json() as Task[]` does **not** verify the data. Validate untrusted data (API responses, forms, `localStorage`) with a runtime schema library such as **Zod**.

TS

```
import { z } from "zod";

const TaskSchema = z.object({
  id: z.number(),
  title: z.string().min(1),
  status: z.enum(["OPEN", "DONE"]),
});
type Task = z.infer<typeof TaskSchema>; // the type is DERIVED from the schema
const TasksSchema = z.array(TaskSchema);

const tasks: Task[] = TasksSchema.parse(await res.json()); // throws if the server sent garbage
const result = TaskSchema.safeParse(input); // { success, data | error }
```

3.9 Error handling patterns

TS

```
try {
  await save();
} catch (err) { // err is `unknown` under strict
  (useUnknownInCatchVariables)
  if (err instanceof HttpError && err.status === 404) return;
  const msg = err instanceof Error ? err.message : String(err);
  throw new Error(`Save failed: ${msg}`, { cause: err }); // preserve original error
}

// Result type: make failure explicit instead of throwing
type Result<T, E = Error> = { ok: true; value: T } | { ok: false; error: E };
function parseAge(s: string): Result<number> {
  const n = Number(s);
  return Number.isInteger(n) && n >= 0 ? { ok: true, value: n } : { ok: false, error: new Error("bad age") };
}
```

4. TypeScript - Advanced

CATEGORY

Language: expert techniques. Template literal types, advanced generics, branded types, **satisfies**, decorators, variance, type-safe APIs, performance and strictness. Level: Advanced.

4.1 Template literal types

TS

```
type Method = "GET" | "POST";
type Path = "/tasks" | "/users";
type Route = `${Method} ${Path}`; // "GET /tasks" | "GET /users" | "POST /tasks" | "POST /users"

type EventName<T extends string> = `on${Capitalize<T>}`;
type ClickEvt = EventName<"click">; // "onClick"

type PathParams<S extends string> =
  S extends `${string}:${infer P}/${infer Rest}` ? P | PathParams<`${Rest}`> :
  S extends `${string}:${infer P}` ? P : never;
type Params = PathParams<"/users/:userId/tasks/:taskId">; // "userId" | "taskId"
```

4.2 The **satisfies** operator

satisfies checks that a value matches a type **without widening** it.

TS

```
type Colors = Record<string, string | number[]>;

const palette = {
  red: [255, 0, 0],
  green: "#00ff00",
} satisfies Colors;

palette.red.map(n => n * 2); // still number[] (with `: Colors` it would be string | number[])
palette.green.toUpperCase(); // still string
```

4.3 Branded (nominal) types

TypeScript is *structural*: a **UserId** and an **OrderId** that are both **string** are interchangeable. Brands prevent mixing.

TS

```
type Brand<T, B extends string> = T & { readonly __brand: B };
type UserId = Brand<string, "UserId">;
type OrderId = Brand<string, "OrderId">;

const UserId = (s: string) => s as UserId;
const OrderId = (s: string) => s as OrderId;

function loadOrder(id: OrderId) { /* ... */ }
loadOrder(OrderId("o-1")); // OK
// loadOrder(UserId("u-1")); // Error: UserId is not assignable to OrderId
```

4.4 Advanced generic patterns

TS

```
// 1) const type parameters (TS 5.0): infer literal types from arguments
function tuple<const T extends readonly unknown[]>(...xs: T): T { return xs; }
const t = tuple("a", 1, true); // readonly ["a", 1, true]

// 2) Builder with accumulating types
class Query<S extends object = {}> {
  select<K extends string>(k: K): Query<S & Record<K, unknown>> { return this as any; }
}

// 3) Function overload replacement with conditional return type
type Fetcher<T extends "one" | "many"> = T extends "one" ? User : User[];
declare function get<T extends "one" | "many">(mode: T): Fetcher<T>;
const one = get("one"); // User
const many = get("many"); // User[]

// 4) Deep partial and paths
type DeepPartial<T> = T extends object ? { [K in keyof T]?: DeepPartial<T[K]> } : T;

// 5) Type-safe event emitter
type Events = { login: { userId: string }; logout: void };
class Emitter<E extends Record<string, unknown>> {
  on<K extends keyof E>(k: K, fn: (p: E[K]) => void) { /* ... */ }
  emit<K extends keyof E>(k: K, ...p: E[K] extends void ? [] : [E[K]]) { /* ... */ }
}
new Emitter<Events>().emit("login", { userId: "1" });
```

4.5 Variance and assignability (why some assignments fail)

Concept	Meaning	Example
Covariance	<code>Dog[]</code> assignable to <code>Animal[]</code> (read positions)	Return types, arrays (unsound for writes)
Contravariance	<code>(a: Animal) => void</code> assignable to <code>(d: Dog) => void</code>	Function parameters under <code>strictFunctionTypes</code>
Invariance	Must match exactly	Both read and write positions
<code>in</code> / <code>out</code> annotations	Explicit variance for generics (TS 4.7)	<code>interface Producer<out T> {}</code>
Excess property check	Object literals must not have unknown keys	<code>const p: Point = { x: 1, y: 2, z: 3 }</code> errors

4.6 Decorators (standard TC39 in TS 5)

Used heavily by NestJS and Angular (which historically used the experimental version enabled with `experimentalDecorators`). Standard decorators are functions that receive the decorated value and a context.

TS

```
function logged<T extends (...a: any[]) => any>(fn: T, ctx: ClassMethodDecoratorContext) {
  return function (this: unknown, ...args: Parameters<T>): ReturnType<T> {
    console.log(`calling ${String(ctx.name)}`, args);
    return fn.apply(this, args);
  } as T;
}

class Service {
  @logged
  work(n: number) { return n * 2; }
}
```

NOTE

NestJS and TypeORM still rely on `experimentalDecorators` + `emitDecoratorMetadata` (needed for DI). Follow your framework's `tsconfig` guidance rather than mixing both styles.

4.7 The `using` keyword (explicit resource management)

TS

```
class Conn implements Disposable {
  [Symbol.dispose]() { console.log("closed"); }
}
{
  using c = new Conn();    // automatically disposed at the end of the block
}                          // like Java try-with-resources
```

4.8 Strictness and safety checklist

Setting / practice	Why
"strict": true	Non-negotiable baseline
"noUncheckedIndexedAccess": true	Forces handling of missing array/object entries
Avoid <code>any</code> ; use <code>unknown</code> and narrow	<code>any</code> silently infects everything it touches
Avoid non-null <code>!</code> and unchecked <code>as</code>	They bypass the compiler; validate instead
ESLint <code>typescript-eslint</code> recommended-type-checked	Catches floating promises, unsafe assignments
<code>Object.keys(o)</code> returns <code>string[]</code>	Use <code>Object.entries</code> typed helpers or narrow keys deliberately
Validate boundaries with Zod / Valibot / ArkType	Types cannot see the network

4.9 Performance and project organisation

- Prefer **interfaces** + `extends` over large chained intersections in hot paths of huge codebases.
- Split projects using **project references** ("`composite`": `true`) and `tsc -b` for monorepos.
- Use `tsc --noEmit` in CI and let **esbuild** / **swc** / **Vite** transpile in dev (fast). Use `--generateTrace` to diagnose slow types.
- Avoid extremely deep recursive conditional types; add explicit return types on exported functions to speed up declaration emit.
- TypeScript's compiler is being ported to native code (the "TypeScript 7" effort) for significantly faster builds; the language you write stays the same.

4.10 Running TypeScript directly

Recent Node versions can execute `.ts` files by **stripping types** (`node app.ts`), supporting erasable syntax only (no `enum`, no parameter properties, no `namespace` unless a transform flag is used). For full features use `tsx`, `ts-node` or build with `tsc` / `tsup`.

4.11 TypeScript vs Java (for backend developers)

Concept	Java	TypeScript
Type system	Nominal (classes must be declared related)	Structural (same shape = compatible)
Generics	Erased at runtime, no primitives	Erased at runtime, very expressive (conditional, mapped)
Null safety	<code>Optional</code> , annotations	<code>strictNullChecks</code> , <code>T undefined</code>
Sealed hierarchies	<code>sealed interface</code> + <code>switch</code> patterns	Discriminated unions + <code>switch</code>
Immutability	<code>record</code> , <code>final</code>	<code>readonly</code> , <code>as const</code> , <code>Readonly<T></code>
Interfaces	Explicit <code>implements</code> required	Implicit satisfaction (duck typing)

Concept	Java	TypeScript
Reflection at runtime	Rich (annotations, classes)	Very limited (types erased); use schemas or decorators
Enums	Full classes	Prefer literal unions
Checked exceptions	Yes	No; <code>catch</code> gives <code>unknown</code>
Runtime	JVM	JavaScript engines (V8, SpiderMonkey, JSC)

5. React - Basics

CATEGORY

Library: component-based UI. JSX, components, props, state, events, lists, conditional rendering and project setup with Vite and TypeScript. Used for single-page apps, dashboards, design systems and as the foundation of Next.js. Level: Beginner.

5.1 What is React and why use it?

React is a JavaScript library for building user interfaces from small, reusable **components**. You describe **what the UI should look like for the current data (state)**; React works out how to update the browser DOM efficiently. This is **declarative** programming, in contrast to imperative DOM code (`document.getElementById(...).innerText = ...`).

TEXT

```
UI = f(state)      Change state -> React re-runs your component function
                   -> compares old and new output (reconciliation) -> patches only the differences in the
                   DOM
```

Idea	Meaning
Component	A function that returns UI
Props	Inputs passed from parent to child (read-only)
State	Data owned by a component that can change over time
Render	Calling your component function to compute UI
Virtual DOM / reconciliation	React compares descriptions of the UI and updates only what changed
One-way data flow	Data flows down via props; events flow up via callbacks

5.2 Create a project (Vite + React + TypeScript)

BASH

```
npm create vite@latest my-app -- --template react-ts
cd my-app
npm install
npm run dev          # http://localhost:5173
npm run build        # production build in dist/
```

TEXT

```
my-app/
  index.html      <- single HTML page with <div id="root">
  src/
    main.tsx      <- mounts React into #root
    App.tsx       <- root component
  components/     hooks/  pages/  lib/  types/
  vite.config.ts  tsconfig.json  package.json
```

TSX

```
// src/main.tsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

5.3 JSX

JSX is syntax that looks like HTML inside JavaScript. It compiles to function calls. Rules:

1. Return **one root element** (use a fragment `<>...</>` if you do not want a wrapper).
2. **Close every tag**: `<img />`, `<input />`.
3. Use **camelCase** attributes: `className`, `htmlFor`, `onClick`, `tabIndex`.
4. Embed JavaScript **expressions** with `{ }` (not statements like `if` or `for`).
5. `style` takes an object: `style={{ color: "red", fontSize: 16 }}`.

TSX

```
export default function Welcome() {
  const name = "Asha";
  const items = ["Learn", "Build", "Ship"];
  return (
    <>
      <h1 className="title">Hello, {name.toUpperCase()}!</h1>
      <p style={{ color: "gray" }}>2 + 2 = {2 + 2}</p>
      <ul>{items.map((t) => <li key={t}>{t}</li>)}</ul>
    </>
  );
}
```

5.4 Components and props

TSX

```
type ButtonProps = {
  label: string;
  variant?: "primary" | "secondary"; // optional with a default below
  disabled?: boolean;
  onClick: () => void;
};

export function Button({ label, variant = "primary", disabled = false, onClick }: ButtonProps) {
  return (
    <button className={`btn btn-${variant}`} disabled={disabled} onClick={onClick}>
      {label}
    </button>
  );
}

// Usage
<Button label="Save" onClick={() => console.log("saved")} />
```

Children and composition

TSX

```
import type { ReactNode } from "react";

function Card({ title, children }: { title: string; children: ReactNode }) {
  return (
    <section className="card">
      <h2>{title}</h2>
      {children} /* whatever is placed between <Card> tags */
    </section>
  );
}

<Card title="Profile"><Avatar /><Bio /></Card>
```

WARNING

Props are **read-only**. Never modify `props.x = ....`. To change data, the owner (parent) updates its state and passes new props.

5.5 State with useState

TSX

```
import { useState } from "react";

export function Counter() {
  const [count, setCount] = useState(0); // [value, setter]

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount((c) => c + 1)}>+1 (safe updater)</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

Key rules of state:

- State updates are **scheduled**, not immediate: `setCount(count + 1)`; `console.log(count)` still prints the old value in that render.
- Use the **updater form** `setX(prev => ...)` when the new value depends on the previous one (and when multiple updates happen in one event).
- **Never mutate** state. Create new objects/arrays.

TSX

```
const [user, setUser] = useState({ name: "Asha", address: { city: "Chennai" } });
// WRONG: user.name = "Ravi"; setUser(user);
setUser({ ...user, name: "Ravi" }); // shallow copy
setUser((u) => ({ ...u, address: { ...u.address, city: "Pune" } })); // nested update

const [todos, setTodos] = useState<Todo[]>([]);
setTodos((t) => [...t, newTodo]); // add
setTodos((t) => t.filter((x) => x.id !== id)); // remove
setTodos((t) => t.map((x) => x.id === id ? { ...x, done: !x.done } : x)); // update
```

5.6 Events

TSX

```
function SearchBox() {
  const [q, setQ] = useState("");

  const onChange = (e: React.ChangeEvent<HTMLInputElement>) => setQ(e.target.value);
  const onSubmit = (e: React.FormEvent<HTMLFormElement>) => {
    e.preventDefault(); // stop the browser page reload
    console.log("search", q);
  };

  return (
    <form onSubmit={onSubmit}>
      <input value={q} onChange={onChange} placeholder="Search..." /> /* controlled input */
      <button type="submit">Go</button>
    </form>
  );
}
```

Common event types	Type
Click	<code>React.MouseEvent<HTMLButtonElement></code>
Input change	<code>React.ChangeEvent<HTMLInputElement></code>
Form submit	<code>React.FormEvent<HTMLFormElement></code>
Key press	<code>React.KeyboardEvent<HTMLInputElement></code>
Focus / blur	<code>React.FocusEvent<HTMLInputElement></code>

5.7 Conditional rendering and lists

TSX

```
function TaskList({ tasks, loading }: { tasks: Task[]; loading: boolean }) {
  if (loading) return <Spinner />; // early return
  if (tasks.length === 0) return <p>No tasks yet.</p>;

  return (
    <ul>
      {tasks.map((t) => (
        <li key={t.id} className={t.done ? "done" : ""}> /* stable unique key! */
          {t.title} {t.done && <span>(done)</span>} /* && short-circuit */
          {t.priority > 3 ? <strong>High</strong> : null} /* ternary */
        </li>
      ))}
    </ul>
  );
}
```

WARNING

Keys must be stable and unique among siblings (a database id). Do not use the array index if items can be reordered, inserted or removed: React will mix up state between items. Beware `{count && <X />}`: when `count` is `0`, React renders the number `0`. Use `count > 0 &&`.

5.8 Lifting state up and thinking in React

When two components need the same data, move the state to their **closest common parent** and pass it down.

TSX

```
function TodoApp() {
  const [todos, setTodos] = useState<Todo[]>([]);
  const add = (title: string) => setTodos((t) => [...t, { id: crypto.randomUUID(), title, done: false }]);
  const toggle = (id: string) => setTodos((t) => t.map((x) => x.id === id ? { ...x, done: !x.done } : x));

  return (
    <>
      <AddTodo onAdd={add} /> /* child -> parent via callback prop */
      <TodoList todos={todos} onToggle={toggle} /> /* parent -> child via props */
      <Summary total={todos.length} done={todos.filter((t) => t.done).length} /> /* derived, not stored */
    </>
  );
}
```

Thinking-in-React steps: (1) break the UI into a component hierarchy, (2) build a static version, (3) find the **minimal** state, (4) decide where state lives, (5) add inverse data flow (callbacks).

PRO TIP

Do not store what you can compute. `done` count, filtered lists and totals should be calculated during render from existing state, not kept in extra state variables that can get out of sync.

6. React - Hooks in Depth

CATEGORY

Library: the hooks API. Every built-in hook with syntax, when to use it, and pitfalls, including the React 19 additions. Level: Basic to Intermediate.

6.1 The rules of hooks

1. Call hooks **only at the top level** of a component or custom hook (not in loops, conditions or nested functions).
2. Call hooks **only from** React function components or other hooks.
3. Custom hook names start with **use**.

The rules exist because React identifies hooks by their **call order** on every render.

6.2 useEffect: synchronising with the outside world

Effects run **after render** to synchronise your component with something external: network, timers, subscriptions, the DOM, browser APIs.

TSX

```
useEffect(() => {
  // setup: runs after render when dependencies changed
  const id = setInterval(() => setSeconds((s) => s + 1), 1000);
  return () => clearInterval(id); // cleanup: runs before the next effect and on unmount
}, []); // dependency array
```

Dependency array	When the effect runs
Omitted	After every render (rarely wanted)
<code>[]</code>	Once after mount (and cleanup on unmount)
<code>[a, b]</code>	After mount and whenever a or b change

TSX

```
// Fetching data with cancellation (simple version; prefer TanStack Query in real apps)
function User({ id }: { id: number }) {
  const [state, setState] = useState<LoadState<User>>>({ status: "loading" });

  useEffect(() => {
    const ctrl = new AbortController();
    setState({ status: "loading" });
    fetch(`/api/users/${id}`, { signal: ctrl.signal })
      .then((r) => { if (!r.ok) throw new Error(String(r.status)); return r.json(); })
      .then((data) => setState({ status: "success", data }))
      .catch((error) => { if (error.name !== "AbortError") setState({ status: "error", error }); });
    return () => ctrl.abort(); // avoids race conditions when id changes quickly
  }, [id]);

  if (state.status === "loading") return <Spinner />;
  if (state.status === "error") return <p>{state.error.message}</p>;
  return <h1>{state.data.name}</h1>;
}
```

WARNING

You might not need an effect. Do not use `useEffect` to (a) derive data from props/state (compute during render), (b) handle user events (use event handlers), (c) reset state when a prop changes (use a **key**). Overusing effects causes extra renders and bugs.

6.3 useRef: mutable value or DOM handle

TSX

```
function FocusInput() {
  const inputRef = useRef<HTMLInputElement>(null); // DOM reference
  const renders = useRef(0); // mutable box that does NOT trigger re-render
  renders.current += 1;

  return (
    <>
      <input ref={inputRef} />
      <button onClick={() => inputRef.current?.focus()}>Focus</button>
    </>
  );
}
```

6.4 useContext: avoid prop drilling

TSX

```
import { createContext, useContext, useState, type ReactNode } from "react";

type Theme = "light" | "dark";
type ThemeCtx = { theme: Theme; toggle: () => void };

const ThemeContext = createContext<ThemeCtx | null>(null);

export function ThemeProvider({ children }: { children: ReactNode }) {
  const [theme, setTheme] = useState<Theme>("light");
  const value = { theme, toggle: () => setTheme((t) => (t === "light" ? "dark" : "light")) };
  return <ThemeContext value={value}>{children}</ThemeContext>; // React 19: no .Provider needed
}

export function useTheme() { // custom hook gives a safe, typed accessor
  const ctx = useContext(ThemeContext);
  if (!ctx) throw new Error("useTheme must be used inside ThemeProvider");
  return ctx;
}
```

WARNING

Every consumer re-renders when the context **value** changes. Split contexts (state vs actions), memoise the value, or use a state library for frequently changing data.

6.5 useReducer: complex state logic

TSX

```
type Action =
  | { type: "add"; title: string }
  | { type: "toggle"; id: string }
  | { type: "remove"; id: string };

function reducer(state: Todo[], action: Action): Todo[] {
  switch (action.type) {
    case "add": return [...state, { id: crypto.randomUUID(), title: action.title, done: false }];
    case "toggle": return state.map((t) => (t.id === action.id ? { ...t, done: !t.done } : t));
    case "remove": return state.filter((t) => t.id !== action.id);
  }
}

const [todos, dispatch] = useReducer(reducer, []);
dispatch({ type: "add", title: "Learn hooks" });
```

Use **useReducer** when state has several sub-values or many transitions. It is the same idea as Redux, at component scope.

6.6 useMemo and useCallback: performance hints

TSX

```
const visible = useMemo(() => todos.filter(t => t.title.includes(query)), [todos, query]); // cache a
  VALUE
const onSelect = useCallback((id: string) => setSelected(id), []); // cache a
  FUNCTION
const Row = memo(function Row({ item, onSelect }: RowProps) { /* ... */ }); // skip
  re-render if props equal
```

Tool	Caches	Use when
<code>useMemo(fn, deps)</code>	A computed value	Expensive computation, or stable object identity for a memoised child or dependency array
<code>useCallback(fn, deps)</code>	A function identity	Passing callbacks to <code>memo</code> children or effect dependencies
<code>memo(Component)</code>	The rendered output	Component re-renders often with the same props

PRO TIP

Do **not** wrap everything in `useMemo/useCallback`. Measure first with React DevTools Profiler. The **React Compiler** (stable, opt-in) memoises automatically at build time, making most manual memoisation unnecessary.

6.7 Other built-in hooks

Hook	Purpose	Example
<code>useId</code>	Stable unique id for accessibility attributes (SSR-safe)	<code>const id = useId(); <label htmlFor={id}></code>
<code>useLayoutEffect</code>	Effect that runs before the browser paints (measure DOM)	Tooltips positioning
<code>useImperativeHandle</code>	Customise the ref a component exposes	Rare; expose <code>focus()</code>
<code>useSyncExternalStore</code>	Subscribe to an external store safely	Libraries (Zustand, Redux use it)
<code>useDebugValue</code>	Label custom hooks in DevTools	Debugging
<code>useTransition</code>	Mark a state update as non-urgent	Keep UI responsive while filtering big lists
<code>useDeferredValue</code>	Defer re-rendering with a lagging copy of a value	Search-as-you-type on heavy lists

TSX

```
const [isPending, startTransition] = useTransition();
const onChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  setInput(e.target.value); // urgent: keep typing responsive
  startTransition(() => setQuery(e.target.value)); // non-urgent: expensive list update can be interrupted
};

const deferredQuery = useDeferredValue(query); // use deferredQuery for the slow list
```

6.8 React 19 hooks and APIs

TSX

```
// use(): read a Promise or Context during render (works with Suspense; can be called conditionally)
import { use, Suspense } from "react";
function Comments({ promise }: { promise: Promise<Comment[]> }) {
  const comments = use(promise); // suspends until resolved
  return <ul>{comments.map((c) => <li key={c.id}>{c.text}</li>)}</ul>;
}
<Suspense fallback={<Spinner />}><Comments promise={commentsPromise} /></Suspense>
// Actions: async functions used for form submissions, with pending/error handled for you
import { useActionState } from "react";
function AddTask() {
  const [error, submit, isPending] = useActionState(async (_prev: string | null, formData: FormData) => {
    const title = String(formData.get("title") ?? "");
    if (!title) return "Title is required";
    await api.createTask({ title });
    return null;
  }, null);

  return (
    <form action={submit}> /* form action accepts a function */
      <input name="title" />
      <button disabled={isPending}>{isPending ? "Saving..." : "Add"}</button>
      {error && <p role="alert">{error}</p>}
    </form>
  );
}

// useOptimistic(): show the expected result instantly, roll back on failure
import { useOptimistic } from "react";
const [optimisticTodos, addOptimistic] = useOptimistic(todos, (cur, t: Todo) => [...cur, t]);

// useFormStatus (react-dom): child of a form reads pending state
import { useFormStatus } from "react-dom";
function SubmitButton() { const { pending } = useFormStatus(); return <button
  disabled={pending}>Save</button>; }
```

Other React 19 changes: **ref is a normal prop** for function components (no `forwardRef`), `<Context value>` replaces `<Context.Provider>`, ref callbacks can return cleanup, `<title>` and `<meta>` can be rendered inside components, and better error reporting and Web Component support.

6.9 Custom hooks: reuse logic, not UI

TSX

```
// useLocalStorage.ts
export function useLocalStorage<T>(key: string, initial: T) {
  const [value, setValue] = useState<T>(() => {
    const raw = localStorage.getItem(key);
    return raw ? (JSON.parse(raw) as T) : initial; // lazy initial state (runs once)
  });
  useEffect(() => { localStorage.setItem(key, JSON.stringify(value)); }, [key, value]);
  return [value, setValue] as const;
}

// useDebounce.ts
export function useDebounce<T>(value: T, ms = 300) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), ms);
    return () => clearTimeout(id);
  }, [value, ms]);
  return debounced;
}

// useOnlineStatus.ts (external store subscription)
export function useOnlineStatus() {
  return useSyncExternalStore(
    (cb) => { window.addEventListener("online", cb); window.addEventListener("offline", cb);
      return () => { window.removeEventListener("online", cb); window.removeEventListener("offline", cb); } },
    () => navigator.onLine,
    () => true, // server snapshot
  );
}
```

6.10 Hooks cheat sheet

Need	Hook
Local UI state	<code>useState</code>
Complex state transitions	<code>useReducer</code>
Share state without props	<code>useContext</code>
Run code after render (sync with outside world)	<code>useEffect</code>
DOM access or mutable non-rendering value	<code>useRef</code>
Cache expensive value / function	<code>useMemo</code> / <code>useCallback</code>
Keep UI responsive during heavy update	<code>useTransition</code> , <code>useDeferredValue</code>
Form submit with pending + error	<code>useActionState</code> , <code>useFormStatus</code>
Instant optimistic UI	<code>useOptimistic</code>
Read a promise during render	<code>use</code>
External store	<code>useSyncExternalStore</code>

7. React - Intermediate

CATEGORY

Library: building real apps. Forms, data fetching, routing, state management options, styling approaches and typing components with TypeScript. Level: Intermediate.

7.1 Forms: controlled vs uncontrolled

Approach	How	Good for
Controlled	Input value lives in React state (value + onChange)	Live validation, dependent fields
Uncontrolled	DOM holds the value; read with FormData or a ref	Simple forms, performance, React 19 form actions
Form library	React Hook Form + Zod	Medium to large forms

TSX

```
// Uncontrolled form with FormData (simple and fast)
function Login({ onLogin }: { onLogin: (email: string, pw: string) => void }) {
  return (
    <form onSubmit={e => {
      e.preventDefault();
      const f = new FormData(e.currentTarget);
      onLogin(String(f.get("email")), String(f.get("password")));
    }}>
      <input name="email" type="email" required />
      <input name="password" type="password" minLength={8} required />
      <button>Sign in</button>
    </form>
  );
}
```

TSX

```
// React Hook Form + Zod: typed, minimal re-renders, schema validation
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { z } from "zod";

const Schema = z.object({
  title: z.string().min(1, "Required").max(100),
  priority: z.coerce.number().int().min(1).max(5),
});
type FormValues = z.infer<typeof Schema>;

export function TaskForm({ onSave }: { onSave: (v: FormValues) => Promise<void> }) {
  const { register, handleSubmit, formState: { errors, isSubmitting } } =
    useForm<FormValues>({ resolver: zodResolver(Schema), defaultValues: { priority: 3 } });

  return (
    <form onSubmit={handleSubmit(onSave)} noValidate>
      <input {...register("title")} aria-invalid={!!errors.title} />
      {errors.title && <p role="alert">{errors.title.message}</p>}
      <input type="number" {...register("priority")} />
      <button disabled={isSubmitting}>Save</button>
    </form>
  );
}
```

7.2 Data fetching: TanStack Query (server state)

Server state (data owned by the backend, cached on the client) is different from **UI state**. TanStack Query handles caching, deduplication, refetching, retries, pagination and optimistic updates so you do not hand-write `useEffect` + `useState` loading logic.

TSX

```
import { QueryClient, QueryClientProvider, useQuery, useMutation, useQueryClient } from
  "@tanstack/react-query";

const queryClient = new QueryClient({ defaultOptions: { queries: { staleTime: 30_000, retry: 1 } } });
// <QueryClientProvider client={queryClient}><App /></QueryClientProvider>

// Read
function useTasks(status?: Status) {
  return useQuery({
    queryKey: ["tasks", { status }], // cache key: changes trigger a refetch
    queryFn: () => api.getTasks({ status }),
    placeholderData: (prev) => prev, // keep old data while loading the new key
  });
}

// Write + invalidate
function useCreateTask() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (dto: CreateTask) => api.createTask(dto),
    onSuccess: () => qc.invalidateQueries({ queryKey: ["tasks"] }), // refetch lists
  });
}

// Optimistic toggle with rollback
function useToggleTask() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (id: number) => api.completeTask(id),
    onMutate: async (id) => {
      await qc.cancelQueries({ queryKey: ["tasks"] });
      const prev = qc.getQueryData<Task[]>(["tasks", {}]);
      qc.setQueryData<Task[]>(["tasks", {}], (old) => old?.map((t) => t.id === id ? { ...t, status: "DONE" } : t));
      return { prev };
    },
    onError: (_e, _id, ctx) => qc.setQueryData(["tasks", {}], ctx?.prev), // rollback
    onSettled: () => qc.invalidateQueries({ queryKey: ["tasks"] }),
  });
}

// In a component
const { data, isPending, isError, error, refetch } = useTasks("OPEN");
```

Concern	Hand-written effects	TanStack Query
Loading / error state	Manual	Built in
Caching	None	Automatic with <code>staleTime</code> / <code>gcTime</code>
Duplicate requests	Yes	Deduplicated
Refetch on focus / reconnect	Manual	Automatic
Pagination / infinite scroll	Manual	<code>useInfiniteQuery</code>
Race conditions	Must handle	Handled

Alternatives: **SWR** (lighter), **RTK Query** (if you already use Redux Toolkit), **Apollo Client** (GraphQL).

7.3 Routing with React Router

TSX

```
import { createBrowserRouter, RouterProvider, Link, NavLink, Outlet, useParams, useNavigate,
  useSearchParams, Navigate } from "react-router";

const router = createBrowserRouter([
  {
    path: "/",
    element: <Layout />, // renders <Outlet /> for children
    errorElement: <ErrorPage />,
    children: [
      { index: true, element: <Home /> },
      { path: "tasks", element: <TaskList /> },
      { path: "tasks/:id", element: <TaskDetail /> },
      { path: "admin", element: <RequireAuth><Admin /></RequireAuth> },
      { path: "*", element: <NotFound /> },
    ],
  },
]);
// <RouterProvider router={router} />

function Layout() {
  return (<
    <nav><NavLink to="/" end>Home</NavLink> <NavLink to="/tasks">Tasks</NavLink></nav>
    <Outlet />
  </>);
}

function TaskDetail() {
  const { id } = useParams<{ id: string }>();
  const navigate = useNavigate();
  const [params, setParams] = useSearchParams(); // ?tab=comments
  return <button onClick={() => navigate(-1)}>Back</button>;
}

function RequireAuth({ children }: { children: ReactNode }) {
  const { user } = useAuth();
  return user ? children : <Navigate to="/login" replace />;
}
```

Routing concepts: **nested routes** with `<Outlet />`, **dynamic segments** `:id`, **search params** for filter/sort/page state (shareable URLs), **lazy routes** for code splitting (`lazy: () => import("./pages/Admin")`), **loaders/actions** for data (framework mode).

7.4 State management: choosing the right tool

Kind of state	Best tool
Local UI (open/closed, input text)	<code>useState</code>
Related transitions inside one component	<code>useReducer</code>
Shared across a subtree, rarely changes (theme, user, locale)	Context
Server data (lists, details)	TanStack Query / SWR
URL state (filters, page, tab)	Router search params
Global client state with frequent updates	Zustand / Jotai / Redux Toolkit
Form state	React Hook Form

TSX

```
// Zustand: tiny global store, no provider needed
import { create } from "zustand";

type CartState = { items: Item[]; add: (i: Item) => void; clear: () => void };
export const useCart = create<CartState>((set) => ({
  items: [],
  add: (i) => set((s) => ({ items: [...s.items, i] })),
  clear: () => set({ items: [] }),
}));

const count = useCart((s) => s.items.length); // selector: re-render only when this value changes
```

TSX

```
// Redux Toolkit: structured, DevTools, larger teams
import { configureStore, createSlice, type PayloadAction } from "@reduxjs/toolkit";

const counter = createSlice({
  name: "counter",
  initialState: { value: 0 },
  reducers: {
    increment: (s) => { s.value += 1; }, // Immer allows "mutating" syntax safely
    addBy: (s, a: PayloadAction<number>) => { s.value += a.payload; },
  },
});
export const store = configureStore({ reducer: { counter: counter.reducer } });
export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

Library	Style	Boilerplate	Best for
Context + useReducer	Built in	Medium	Small/medium apps, low-frequency state
Zustand	Hook-based store	Very low	Most apps needing global client state
Jotai / Recoil-style atoms	Atomic state	Low	Fine-grained derived state
Redux Toolkit	Actions + reducers	Higher	Large teams, time-travel debugging, strict patterns
MobX	Observable objects	Low	OOP style, automatic tracking

7.5 Styling options

Approach	Example	Notes
Plain CSS / CSS Modules	<code>import s from "./Button.module.css"; className={s.btn}</code>	Scoped class names, zero runtime
Tailwind CSS	<code>className="rounded bg-blue-600 px-4 py-2 text-white"</code>	Utility-first, fast, covered in Volume 3
CSS-in-JS (styled-components, Emotion)	<code>const Btn = styled.button</code>	Runtime cost; less used with Server Components
Zero-runtime CSS-in-JS (vanilla-extract, Panda)	Type-safe styles compiled at build time	Good with RSC
Component libraries	MUI, Chakra, Mantine, shadcn/ui (Radix + Tailwind)	Faster start; shadcn/ui copies code into your repo

7.6 Typing React components

TSX

```
// Props extending native element props
type InputProps = React.ComponentPropsWithoutRef<"input"> & { label: string; error?: string };

export function Field({ label, error, id, ...rest }: InputProps) {
  const auto = useId(); const inputId = id ?? auto;
  return (
    <div>
      <label htmlFor={inputId}>{label}</label>
      <input id={inputId} aria-invalid={!!error} {...rest} />
      {error && <span role="alert">{error}</span>}
    </div>
  );
}

// ref as a prop (React 19) - no forwardRef required
function FancyInput({ ref, ...props }: React.ComponentProps<"input">) { return <input ref={ref} {...props}>
```

```
    /> }

// Generic component
type ListProps<T> = { items: T[]; render: (item: T) => ReactNode; keyOf: (item: T) => string | number };
function List<T>({ items, render, keyOf }: ListProps<T>) {
  return <ul>{items.map((i) => <li key={keyOf(i)}>{render(i)}</li>)}</ul>;
}
<List items={tasks} keyOf={(t) => t.id} render={(t) => t.title} /> // T inferred as Task
// Discriminated-union props: make invalid combinations impossible
type ButtonProps =
  | { as: "button"; onClick: () => void; href?: never }
  | { as: "link"; href: string; onClick?: never };
```

Type	Use
<code>ReactNode</code>	Anything renderable (children)
<code>ReactElement</code>	A JSX element
<code>React.FC<Props></code>	Optional; plain functions are preferred
<code>ComponentProps<typeof X></code>	Props of another component
<code>React.Dispatch<React.SetStateAction<T>></code>	Type of a <code>useState</code> setter passed as prop
<code>CSSProperties</code>	The <code>style</code> object

8. React - Advanced

CATEGORY

Library: performance, patterns and architecture. Rendering internals, memoisation, code splitting, Suspense, error boundaries, design patterns, testing, accessibility and React Server Components. Level: Advanced.

8.1 How rendering really works

1. **Trigger:** state changes (or a parent re-renders, or context changes).
2. **Render:** React calls your component functions to produce a new tree (**must be pure**: same props and state give same output; no side effects).
3. **Commit:** React applies the minimal DOM changes; then runs layout effects and effects.
 - A component re-renders when its **state** changes, its **parent** re-renders (even if props are equal, unless **memo**), or a **context** it reads changes.
 - **Reconciliation** compares element type and **key** at each position. Same type and key: keep the instance and state. Different: unmount and remount (state lost).
 - **Batching:** React 18+ batches multiple state updates in any event, timeout or promise into one render.

TSX

```
// Reset a component's state by changing its key
<Profile key={userId} userId={userId} /> // new user -> fresh state, no effect needed
```

8.2 Performance optimisation playbook

Symptom	Technique
Whole tree re-renders on every keystroke	Move state down (colocate) or split the component
Expensive child re-renders with same props	<code>React.memo</code> + stable props (<code>useCallback/useMemo</code>) or the React Compiler
Slow long lists (thousands of rows)	Virtualisation (<code>@tanstack/react-virtual</code> , <code>react-window</code>)
Slow typing with heavy filtering	<code>useDeferredValue</code> / <code>useTransition</code>
Large bundle	Code splitting: <code>React.lazy</code> , dynamic <code>import()</code> , route-level splitting
Context causes wide re-renders	Split contexts, memoise value, or use selectors (Zustand)
Wasteful network calls	TanStack Query caching, debounce input
Unnecessary re-fetch on render	Move fetching out of render/effects into a data library or server

TSX

```
// Code splitting + Suspense
const Chart = lazy(() => import("./Chart")); // loaded only when first rendered

function Dashboard() {
  return (
    <Suspense fallback={<Skeleton />}>
      <Chart />
    </Suspense>
  );
}

// memo with stable props
const Row = memo(function Row({ task, onToggle }: { task: Task; onToggle: (id: number) => void }) {
  return <li onClick={() => onToggle(task.id)}>{task.title}</li>;
});
const onToggle = useCallback((id: number) => toggle.mutate(id), [toggle.mutate]);
```

TSX

```
// Virtualised list: only visible rows exist in the DOM
const parentRef = useRef<HTMLDivElement>(null);
const rv = useVirtualizer({ count: rows.length, getScrollElement: () => parentRef.current, estimateSize: ()
  => 40 });

<div ref={parentRef} style={{ height: 500, overflow: "auto" }}>
  <div style={{ height: rv.getTotalSize(), position: "relative" }}>
    {rv.getVirtualItems().map((v) => (
      <div key={v.key} style={{ position: "absolute", top: 0, transform: `translateY(${v.start}px)`, height:
        v.size }}>
        {rows[v.index].name}
      </div>
    ))}
  </div>
</div>
```

PRO TIP

Order of optimisation: (1) measure with the React DevTools Profiler, (2) reduce work (state placement, derived data), (3) reduce bundle (splitting), (4) memoise only hot paths, (5) enable the React Compiler.

8.3 Error boundaries and Suspense

TSX

```
// Error boundaries are still class-based (or use the react-error-boundary library)
import { ErrorBoundary } from "react-error-boundary";

<ErrorBoundary
  fallbackRender={({ error, resetErrorBoundary }) => (
    <div role="alert"><p>Something broke: {error.message}</p><button
      onClick={resetErrorBoundary}>Retry</button></div>
  )}
  onError={(e) => reportToSentry(e)}
>
  <Suspense fallback={<Spinner />}>
    <Widget />
  </Suspense>
</ErrorBoundary>
```

Error boundaries catch errors in **rendering** and lifecycle. They do **not** catch errors in event handlers, async code or the server; handle those with **try/catch**. **Suspense** shows a fallback while a child waits (lazy code, **use(promise)**, framework data).

8.4 Component design patterns

Pattern	Idea	Example
Custom hook	Extract stateful logic	<code>useDebounce</code> , <code>useTasks</code>
Compound components	Related components share implicit state via context	<code><Tabs><Tabs.List/><Tabs.Panel/></Tabs></code>
Controlled + uncontrolled	Support both <code>value</code> and <code>defaultValue</code>	Inputs, dialogs
Render props	Pass a function that returns UI	<code><Mouse>{pos => <Dot {...pos}/>}</Mouse></code> (mostly replaced by hooks)
Higher-order component	Function that wraps a component	<code>withAuth(Page)</code> (legacy, still seen)
Container / presentational	Data logic separated from UI	Hooks now replace containers
Slots via props	Accept elements as props	<code><Layout header={<Nav/>} footer={<Foot/>}/></code>
Polymorphic <code>as</code> prop	Render different element types	<code><Text as="h1"></code>

TSX

```
// Compound components
const TabsCtx = createContext<{ active: string; setActive: (v: string) => void } | null>(null);

export function Tabs({ defaultValue, children }: { defaultValue: string; children: ReactNode }) {
  const [active, setActive] = useState(defaultValue);
  return <TabsCtx value={{ active, setActive }}><div>{children}</div></TabsCtx>;
}

Tabs.Trigger = function Trigger({ value, children }: { value: string; children: ReactNode }) {
  const c = use(TabsCtx)!;
  return <button role="tab" aria-selected={c.active === value} onClick={() =>
    c.setActive(value)}>{children}</button>;
};

Tabs.Panel = function Panel({ value, children }: { value: string; children: ReactNode }) {
  return use(TabsCtx)!.active === value ? <div role="tabpanel">{children}</div> : null;
};
```

8.5 Project structure and architecture

TEXT

```
src/
  app/           providers, router, global layout
  features/      feature folders (recommended)
    tasks/
      api.ts      <- fetch functions + zod schemas
      hooks.ts    <- useTasks, useCreateTask
      components/ <- TaskList, TaskForm (feature-specific UI)
      types.ts
      index.ts    <- public API of the feature
    components/  shared UI (Button, Modal, Field)
    hooks/        shared hooks
    lib/           utilities, api client, constants
```

Guidelines: organise by **feature**, not only by file type; keep components small; colocate state near where it is used; expose a feature's public API through `index.ts`; forbid feature-to-feature deep imports (ESLint `no-restricted-imports`).

8.6 Testing React

TSX

```
// Vitest + React Testing Library: test what the user sees and does
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { describe, it, expect, vi } from "vitest";

describe("<TaskForm />", () => {
  it("submits a valid task", async () => {
    const onSave = vi.fn().mockResolvedValue(undefined);
    render(<TaskForm onSave={onSave} />);

    await userEvent.type(screen.getByRole("textbox", { name: /title/i }), "Write docs");
    await userEvent.click(screen.getByRole("button", { name: /save/i }));

    expect(onSave).toHaveBeenCalledWith({ title: "Write docs", priority: 3 }, expect.anything());
  });
  it("shows validation error", async () => {
    render(<TaskForm onSave={vi.fn()} />);
    await userEvent.click(screen.getByRole("button", { name: /save/i }));
    expect(await screen.findByRole("alert")).toHaveTextContent(/required/i);
  });
});
```

Test type	Tool	Purpose
Unit (functions, hooks)	Vitest, renderHook	Logic without UI
Component	React Testing Library + user-event	Behaviour through the accessible UI
API mocking	MSW (Mock Service Worker)	Realistic network mocks shared by tests and dev
End-to-end	Playwright / Cypress	Real browser, critical user journeys
Visual	Storybook + Chromatic	UI regression

PRO TIP

Query by **role and accessible name** (**getByRole**), not by class names or test ids. It tests accessibility and behaviour at the same time.

8.7 Accessibility (a11y) essentials

- Use **semantic HTML** first: **<button>**, **<nav>**, **<main>**, **<label>**, headings in order. A **div onClick** is not keyboard accessible.
- Every input needs a **label**; every image needs meaningful **alt** (or **alt=""** if decorative).
- Manage **focus** for dialogs and route changes; ensure visible focus outlines.
- Use ARIA only when native HTML cannot do the job (**aria-live** for status messages, **aria-expanded** for disclosure).
- Ensure colour contrast (WCAG AA 4.5:1 for text) and support keyboard navigation for every interaction.
- Use tested primitives (Radix UI, React Aria, Headless UI) for menus, dialogs, comboboxes.

8.8 React Server Components (RSC) - the next paradigm

Server Components render **on the server** (during build or request), can **await** data directly, and send **no JavaScript** for themselves to the browser. **Client Components** (marked with **"use client"**) hydrate and handle interactivity.

TSX

```
// Server Component (default in Next.js App Router): async, direct DB/API access, zero client JS
export default async function TasksPage() {
  const tasks = await db.task.findMany(); // runs on the server only
  return (
    <main>
      <h1>Tasks</h1>
      <TaskList tasks={tasks} /> /* server-rendered */
      <AddTaskButton /> /* client island below */
    </main>
  );
}

// AddTaskButton.tsx
"use client"; // this file and its imports run in the browser
import { useState } from "react";
export function AddTaskButton() { const [open, setOpen] = useState(false); /* ... */ }
```

	Server Component	Client Component
Runs on	Server	Browser (also pre-rendered on server)
Can use <code>useState</code> , <code>useEffect</code> , event handlers	No	Yes
Can access DB / secrets / file system	Yes	No
Adds JavaScript to the bundle	No	Yes
Marked by	Default (no directive)	" <code>use client</code> " at the top of the file

Server Actions ("`use server`") let a form or button call a server function directly. RSC needs a framework (Next.js, React Router framework mode, Waku); details and full examples are in **Volume 3 (Next.js)**.

8.9 React Compiler

The React Compiler is a build-time tool (Babel/SWC plugin) that **automatically memoises** components and hooks by analysing your code, provided you follow the Rules of React (pure render, no mutation of props/state). It is stable and opt-in; when adopted, you rarely need `useMemo`, `useCallback` or `memo`.

8.10 Common React mistakes

Mistake	Consequence	Fix
Mutating state	UI does not update	Create new objects/arrays
Missing or index <code>key</code>	State mix-ups in dynamic lists	Stable unique ids
<code>useEffect</code> for derived data	Extra renders, stale values	Compute during render
Missing effect cleanup	Leaks, race conditions	Return a cleanup function; abort fetches
Stale closures in effects/timers	Old values used	Correct dependencies, updater form
Defining components inside components	Remounts every render, state lost	Define at module level
Overusing Context for fast-changing data	Wide re-renders	State library with selectors
Storing server data in <code>useState</code> manually	Duplicate caches, no invalidation	TanStack Query
Ignoring <code>react-hooks/exhaustive-deps</code> lint	Subtle bugs	Fix the dependency, do not silence the rule
Copying props into state	State never updates when props change	Derive, or use a <code>key</code> to reset

9. Comparisons: UI Frameworks and Rendering Strategies

CATEGORY

Decision chapter. React compared with Angular, Vue and Svelte; client-side vs server-side rendering; and how TypeScript fits each. Level: All levels.

9.1 React vs Angular vs Vue vs Svelte

Criteria	React	Angular	Vue 3	Svelte 5
Type	Library (UI only)	Full framework	Progressive framework	Compiler-based framework
Language	JS/TS + JSX	TypeScript (mandatory) + templates	JS/TS + SFC templates	JS/TS + <code>.svelte</code> files
Data flow	One-way	One-way (two-way with <code>ngModel</code>)	One-way, <code>v-model</code> sugar	One-way, <code>bind</code> : sugar
Reactivity	Re-render + hooks	Signals (modern) + RxJS + zones	Fine-grained refs/proxy	Runes (<code>\$state</code>), compiled
State management	Zustand / Redux / Context (choose)	Services + RxJS/NgRx/Signals	Pinia	Built-in stores / runes
Routing	React Router / TanStack Router / Next.js	Built in	Vue Router	SvelteKit
Meta-framework	Next.js , Remix / React Router	Angular Universal / SSR built in	Nuxt	SvelteKit
Learning curve	Medium (hooks, ecosystem choices)	Steep (DI, modules, RxJS, decorators)	Gentle	Gentle
Bundle size (typical)	Medium	Larger	Small-medium	Smallest
Runtime performance	Very good (Compiler helps)	Good (signals improving)	Very good	Excellent
Ecosystem and jobs	Largest	Large (enterprise)	Large (Asia, Europe)	Growing
Opinionated structure	Low	Very high	Medium	Medium
Best for	Products needing flexibility, huge ecosystem, mobile via React Native	Large enterprise teams wanting one standard	Teams wanting simplicity and speed	Small bundles, performance-sensitive sites
Comparable Java/Spring mindset	Library you assemble	Closest to Spring (DI, structure)	In between	Minimal

PRO TIP

Choosing: for hiring, ecosystem and full-stack (Next.js) synergy pick **React**. For a large enterprise team used to Spring-style structure consider **Angular**. For quick delivery by a small team **Vue**. For tiny bundles and joyful DX **Svelte**.

9.2 Same component in four frameworks (counter)

TSX

```
// React
function Counter() {
  const [n, setN] = useState(0);
  return <button onClick={() => setN(n + 1)}>Clicked {n}</button>;
}
```

HTML

```
<!-- Vue 3 (script setup) -->
<script setup lang="ts">
import { ref } from "vue";
const n = ref(0);
</script>
<template><button @click="n++">Clicked {{ n }}</button></template>
```

HTML

```
<!-- Svelte 5 -->
<script lang="ts">
  let n = $state(0);
</script>
<button onclick={() => n++}>Clicked {n}</button>
```

TS

```
// Angular (standalone component with signals)
@Component({
  selector: "app-counter",
  standalone: true,
  template: `<button (click)="n.set(n() + 1)">Clicked {{ n() }}</button>`,
})
export class CounterComponent { n = signal(0); }
```

9.3 Rendering strategies

Strategy	Where HTML is built	Pros	Cons	Typical tool
CSR (Client-Side Rendering)	Browser, after JS loads	Simple hosting (static files), rich interactivity	Slower first paint, weak SEO without extra work	React + Vite
SSR (Server-Side Rendering)	Server, per request	Fast first paint, good SEO, personalised	Server cost, more complexity	Next.js, Remix
SSG (Static Site Generation)	Build time	Fastest, cheap CDN hosting	Rebuild for changes; not for per-user data	Next.js, Astro
ISR (Incremental Static Regeneration)	Build + background refresh	Static speed with fresh data	Cache invalidation thinking	Next.js
Streaming SSR + RSC	Server, in chunks	Fast shell, progressive content, less JS	New mental model	Next.js App Router
Islands	Static HTML + small interactive parts	Very little JS	Cross-island state is harder	Astro

TEXT

```
CSR: Browser -> empty HTML + big JS bundle -> JS runs -> fetch data -> UI appears
SSR: Browser -> server renders full HTML -> HTML visible fast -> JS hydrates -> interactive
SSG: Build time renders HTML -> CDN serves instantly -> JS hydrates
```

NOTE

Use plain React + Vite for internal tools, dashboards behind login, and embedded widgets where SEO does not matter. **Use Next.js** (Volume 3) when you need SEO, fast first load, server-side data access or a full-stack setup.

9.4 State management: quick comparison (React ecosystem)

Criteria	Context + hooks	Zustand	Redux Toolkit	TanStack Query	Jotai
Purpose	Share values	Global client state	Global client state	Server state	Atomic state
Setup effort	None	Tiny	Medium	Small	Tiny
Re-render control	Weak	Strong (selectors)	Strong (selectors)	Strong	Strong
DevTools	Basic	Yes (middleware)	Excellent	Excellent	Yes
Learning curve	Low	Low	Medium	Medium	Low
Use it for	Theme, auth user	Cart, UI preferences	Very complex client logic	API data	Derived, fine-grained state

9.5 Build tools comparison

Tool	Role	Notes
Vite	Dev server + bundler	Instant start via native ESM, Rollup/Rolldown builds; default for React SPA
Webpack	Bundler	Legacy standard; powerful but slower and complex
Turbopack	Bundler (Rust)	Default in Next.js 16
esbuild / swc	Compilers/transpilers	Very fast; used inside other tools
Parcel	Zero-config bundler	Simple projects
tsup / tsdown	Library bundlers	Publishing TypeScript packages

9.6 Language-level comparison for full-stack developers

Task	Java + Spring (Vol 1)	TypeScript + Node (Vol 1)	TypeScript + React (this volume)
Define a data shape	<code>record TaskResponse(...)</code>	<code>interface Task {...}</code> / Zod schema	Same <code>Task</code> type shared with backend
Validate input	<code>@Valid</code> + annotations	Zod schema	Zod schema + React Hook Form
Loop and transform	<code>stream().map()</code>	<code>array.map()</code>	<code>array.map()</code> returning JSX
Async call	<code>RestClient</code> , <code>CompletableFuture</code>	<code>await fetch/axios</code>	<code>useQuery</code> , <code>await fetch</code>
Dependency injection	Constructor injection	Manual / NestJS	Context + hooks
Unit test	JUnit + Mockito	Vitest + <code>vi.mock</code>	Vitest + Testing Library
State machine	<code>enum</code> + <code>switch</code>	Discriminated union	Discriminated union in state / <code>useReducer</code>

10. Full Example: Task Manager UI (TypeScript + React)

CATEGORY

Worked project. A typed React front end for the Task API built in Volume 1 (Spring Boot or Node.js): list, filter, create, complete and delete tasks with validation, caching and optimistic updates. Level: Intermediate to Advanced.

10.1 Install

BASH

```
npm create vite@latest task-ui -- --template react-ts && cd task-ui
npm i @tanstack/react-query react-hook-form @hookform/resolvers zod react-router
npm i -D vitest @testing-library/react @testing-library/user-event jsdom msw
```

Proxy API calls in dev so the browser sees one origin (no CORS during development):

TS

```
// vite.config.ts
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
export default defineConfig({
  plugins: [react()],
  server: { proxy: { "/api": "http://localhost:8080" } }, // Spring Boot (or 3000 for Express)
});
```

10.2 Types and API layer (shared contract)

TS

```
// src/features/tasks/api.ts
import { z } from "zod";

export const TaskSchema = z.object({
  id: z.number(),
  title: z.string(),
  priority: z.number(),
  status: z.enum(["OPEN", "DONE"]),
});
export type Task = z.infer<typeof TaskSchema>;

const PageSchema = z.object({
  content: z.array(TaskSchema),
  totalElements: z.number(),
  totalPages: z.number(),
});
export const CreateTaskSchema = z.object({
  title: z.string().trim().min(1, "Title is required").max(100),
  priority: z.coerce.number().int().min(1).max(5),
});
export type CreateTask = z.infer<typeof CreateTaskSchema>;

class ApiError extends Error {
  constructor(public status: number, message: string) { super(message); }
}

async function request<T>(url: string, schema: z.ZodType<T> | null, init?: RequestInit): Promise<T> {
  const res = await fetch(url, {
    ...init,
    headers: { "Content-Type": "application/json", ...init?.headers },
  });
}
```

```

    if (!res.ok) throw new ApiError(res.status, (await res.json().catch(() => null))?.title ?? res.statusText);
    if (res.status === 204 || !schema) return undefined as T;
    return schema.parse(await res.json()); // runtime validation of server data
  }

export const api = {
  list: (status?: string, page = 0) =>
    request(`/api/tasks?page=${page}&size=10${status ? `&status=${status}` : ""}`, PageSchema),
  create: (dto: CreateTask) =>
    request("/api/tasks", TaskSchema, { method: "POST", body: JSON.stringify(dto) }),
  complete: (id: number) => request(`/api/tasks/${id}/complete`, TaskSchema, { method: "PATCH" }),
  remove: (id: number) => request(`/api/tasks/${id}`, null, { method: "DELETE" }),
};

```

10.3 Query hooks

TS

```

// src/features/tasks/hooks.ts
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import { api } from "../api";

export const taskKeys = { all: ["tasks"] as const, list: (s?: string, p = 0) => ["tasks", { s, p }] as const };

export const useTasks = (status?: string, page = 0) =>
  useQuery({ queryKey: taskKeys.list(status, page), queryFn: () => api.list(status, page),
    placeholderData: (prev) => prev });

export function useTaskMutations() {
  const qc = useQueryClient();
  const invalidate = () => qc.invalidateQueries({ queryKey: taskKeys.all });
  return {
    create: useMutation({ mutationFn: api.create, onSuccess: invalidate }),
    complete: useMutation({ mutationFn: api.complete, onSuccess: invalidate }),
    remove: useMutation({ mutationFn: api.remove, onSuccess: invalidate }),
  };
}

```

10.4 Components

TSX

```

// src/features/tasks/TaskForm.tsx
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { CreateTaskSchema, type CreateTask } from "../api";
import { useTaskMutations } from "../hooks";

export function TaskForm() {
  const { create } = useTaskMutations();
  const { register, handleSubmit, reset, formState: { errors } } =
    useForm<CreateTask>({ resolver: zodResolver(CreateTaskSchema), defaultValues: { priority: 3 } });

  return (
    <form onSubmit={handleSubmit((v) => create.mutate(v, { onSuccess: () => reset() }))} noValidate>
      <input placeholder="New task" {...register("title")} aria-invalid={!!errors.title} />
      {errors.title && <p role="alert">{errors.title.message}</p>}
      <select {...register("priority")}>{[1, 2, 3, 4, 5].map((n) => <option key={n}>{n}</option>)}</select>
      <button disabled={create.isPending}>{create.isPending ? "Adding..." : "Add"}</button>
      {create.isError && <p role="alert">{create.error.message}</p>}
    </form>
  );
}

```

TSX

```
// src/features/tasks/TaskList.tsx
import { useState } from "react";
import { useTasks, useTaskMutations } from "../hooks";

export function TaskList() {
  const [status, setStatus] = useState<string | undefined>();
  const [page, setPage] = useState(0);
  const { data, isPending, isError, error, isPlaceholderData } = useTasks(status, page);
  const { complete, remove } = useTaskMutations();

  if (isPending) return <p>Loading...</p>;
  if (isError) return <p role="alert">Failed: {error.message}</p>;

  return (
    <section>
      <select value={status ?? ""} onChange={(e) => { setStatus(e.target.value || undefined); setPage(0); }}>
        <option value="">All</option><option value="OPEN">Open</option><option value="DONE">Done</option>
      </select>

      <ul style={{ opacity: isPlaceholderData ? 0.6 : 1 }}>
        {data.content.map((t) => (
          <li key={t.id}>
            <span style={{ textDecoration: t.status === "DONE" ? "line-through" : "none" }}>{t.title}</span>
            {t.status === "OPEN" && <button onClick={() => complete.mutate(t.id)}>Done</button>}
            <button onClick={() => remove.mutate(t.id)} aria-label={`Delete ${t.title}`}>x</button>
          </li>
        ))}
      </ul>

      <button disabled={page === 0} onClick={() => setPage((p) => p - 1)}>Prev</button>
      <span> {page + 1} / {data.totalPages} </span>
      <button disabled={page + 1 >= data.totalPages} onClick={() => setPage((p) => p + 1)}>Next</button>
    </section>
  );
}
```

TSX

```
// src/App.tsx
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { TaskForm } from "../features/tasks/TaskForm";
import { TaskList } from "../features/tasks/TaskList";

const qc = new QueryClient();
export default function App() {
  return (
    <QueryClientProvider client={qc}>
      <main><h1>Tasks</h1><TaskForm /><TaskList /></main>
    </QueryClientProvider>
  );
}
```

10.5 What this example demonstrates

- **One schema, two uses:** Zod validates server responses and form input; TypeScript types are inferred from it.
- **Server state vs UI state:** TanStack Query owns the tasks; `useState` owns the filter and page.
- **Feature folder** with `api.ts`, `hooks.ts`, components: easy to test and move.
- **Accessibility basics:** labels, `role="alert"`, `aria-label` on icon buttons.
- Works unchanged against **Spring Boot or Node.js** because both implement the same API contract (Volume 1, Chapter 12).

11. Interview Q&A, Cheat Sheet and Roadmap

CATEGORY

Revision chapter. The most frequently asked interview questions on TypeScript and React, quick-reference cards and a learning path. Level: All levels.

11.1 TypeScript interview questions

Question	Short answer
<code>type</code> vs <code>interface</code> ?	Both describe shapes. Interfaces can merge and use <code>extends</code> ; types can express unions, tuples, mapped and conditional types
<code>any</code> vs <code>unknown</code> vs <code>never</code> ?	<code>any</code> disables checks; <code>unknown</code> is the safe top type requiring narrowing; <code>never</code> is the empty type (unreachable / impossible)
What are generics?	Type parameters that keep type relationships, e.g. <code>function id<T>(x: T): T</code>
What is structural typing?	Compatibility is decided by shape, not by declared name
Explain union vs intersection	<code>A B</code> is either; <code>A & B</code> has all members of both
What is a discriminated union?	Union with a common literal field so <code>switch</code> narrows to one variant
What does <code>strict</code> include?	<code>strictNullChecks</code> , <code>noImplicitAny</code> , <code>strictFunctionTypes</code> , <code>strictPropertyInitialization</code> , etc.
<code>readonly</code> vs <code>const</code> ?	<code>const</code> binds a variable; <code>readonly</code> restricts a property; <code>as const</code> makes a literal deeply readonly
What is type erasure?	Types are removed at compile time; nothing exists at runtime
<code>as</code> vs <code>satisfies</code> ?	<code>as</code> asserts (can lie); <code>satisfies</code> checks against a type while keeping the precise inferred type
Utility types you use?	<code>Partial</code> , <code>Pick</code> , <code>Omit</code> , <code>Record</code> , <code>ReturnType</code> , <code>Awaited</code>
How to type <code>catch (e)</code> ?	<code>unknown</code> ; narrow with <code>instanceof Error</code>
What is declaration merging?	Multiple declarations of the same interface/namespace combine into one
Enums vs union literals?	Prefer literal unions or <code>as const</code> objects; enums add runtime code and quirks

11.2 React interview questions

Question	Short answer
What is the virtual DOM / reconciliation?	React builds a tree of UI descriptions, diffs it against the previous one and commits minimal DOM changes
Props vs state?	Props are inputs from the parent (read-only); state is owned data that changes and triggers re-render
Why keys in lists?	Identify items across renders so React preserves the right state and DOM
Controlled vs uncontrolled inputs?	Controlled: React state is the source of truth. Uncontrolled: the DOM holds the value
<code>useEffect</code> dependency array?	Controls when the effect re-runs; must list every reactive value used inside
<code>useMemo</code> vs <code>useCallback</code> ?	Memoise a value vs memoise a function identity

Question	Short answer
What causes a re-render?	State change, parent re-render, context change
How to avoid unnecessary re-renders?	Colocate state, memo , stable props, selectors, React Compiler, virtualisation
What is prop drilling and how to avoid it?	Passing props through many layers; use composition, Context or a state library
Client vs Server Components?	Server: render on server, no client JS, can access data; Client: interactive, uses hooks
What is hydration?	Attaching event handlers and state to server-rendered HTML in the browser
What are custom hooks?	Functions starting with use that reuse stateful logic
How do you handle errors?	Error boundaries for rendering errors; try/catch for events and async; Query error states
What is Suspense?	Declarative loading fallback for lazy code and data that suspends
What changed in React 19?	Actions, use , useActionState , useOptimistic , ref as prop, context as provider, Server Components stable

11.3 Cheat sheet: TypeScript syntax

Need	Syntax
Optional property	<code>name?: string</code>
Union	<code>"a" "b"</code>
Intersection	<code>A & B</code>
Array / tuple	<code>string[], [string, number]</code>
Function type	<code>(a: number) => string</code>
Generic function	<code>function f<T>(x: T): T</code>
Constraint	<code><T extends { id: number }></code>
Keys of type	<code>keyof T</code>
Type from value	<code>typeof value</code>
Indexed access	<code>T["key"], T[number]</code>
Mapped type	<code>{ [K in keyof T]: T[K] }</code>
Conditional	<code>T extends U ? X : Y</code>
Infer	<code>T extends Promise<infer R> ? R : T</code>
Non-null assertion	<code>value! (avoid)</code>
Type assertion	<code>value as Type (avoid when possible)</code>
Type-only import	<code>import type { X } from "./x"</code>

11.4 Cheat sheet: React patterns

Goal	Pattern
Toggle boolean	<code>setOpen((o) => !o)</code>
Add / remove / update in list	spread, filter , map returning new array
Fetch on mount	TanStack Query useQuery (not raw effect)
Debounce search	useDebounce (value, 300) then query with debounced value

Goal	Pattern
Persist state	<code>useLocalStorage</code> custom hook
Reset child state	Change its <code>key</code>
Share state up	Lift to common parent
Avoid prop drilling	Composition (<code>children</code>) first, then Context
Heavy list	Virtualisation
Split code	<code>lazy(() => import(...))</code> + <code>Suspense</code>

11.5 Learning roadmap

Stage	TypeScript	React	Project
1. Foundations	ES6+, primitives, arrays, objects, functions	JSX, components, props, <code>useState</code> , events	Counter, todo list
2. Core	Interfaces, unions, generics, narrowing	<code>useEffect</code> , lists, forms, lifting state	Weather app with an API
3. Real apps	Utility types, Zod inference, async typing	Router, TanStack Query, React Hook Form	Task manager UI (Ch 10)
4. Quality	Strict config, ESLint type-aware rules	Vitest, Testing Library, MSW, a11y	Tested feature with 80% coverage
5. Performance	Project references, type performance	Memo, Suspense, code splitting, virtualisation	Dashboard with big tables
6. Advanced	Conditional/mapped/template types, branded types	Custom hook libraries, patterns, Compiler, RSC	Design system or component library
7. Full-stack	Shared types across client and server	Next.js App Router (Volume 3)	Production full-stack app

PRO TIP

Practice loop: read a concept, type the example from scratch (do not paste), break it on purpose, fix it, then rebuild it with a small twist. Enable `strict` from day one so the compiler teaches you.